

Distributed Job Scheduler

Design and Implementation Report

Submitted by: Sriram Sivaraman

Registration Number: RA2311056010159

Stack: Node.js, TypeScript, PostgreSQL, React

1. Introduction

This project is a distributed job scheduler: a system where users submit background jobs through a REST API, and a pool of worker processes picks those jobs up and runs them. It is similar in spirit to tools such as UiPath Orchestrator, BullMQ or Sidekiq, but built from scratch so I could focus on the parts that are genuinely difficult — running each job exactly once across many workers, retrying failures in a controlled way, and recovering cleanly when a worker crashes mid-job.

The single most important decision in the whole system is to use **PostgreSQL as the queue itself**, instead of adding Redis or a separate message broker. Jobs are claimed with `SELECT ... FOR UPDATE SKIP LOCKED`, which lets many workers pull from the same queue at the same time without ever handing the same job to two workers. This keeps the design small — one database is the source of truth and the queue — while still giving the correctness guarantees the task asks for.

The system supports authentication and projects, configurable queues (priority, concurrency limit, pause/resume, retry policy), five kinds of job (immediate, delayed, scheduled, recurring via cron, and batch), a full job lifecycle with retries and a Dead Letter Queue, and a web dashboard for monitoring everything.

2. System architecture

The backend is a single codebase that runs in four different modes — an API server, a scheduler, a reaper, and a worker. Splitting it this way means the API and the workers can be scaled independently, but they all share the same database access code and job logic. The web dashboard talks only to the API; every background process talks only to PostgreSQL.

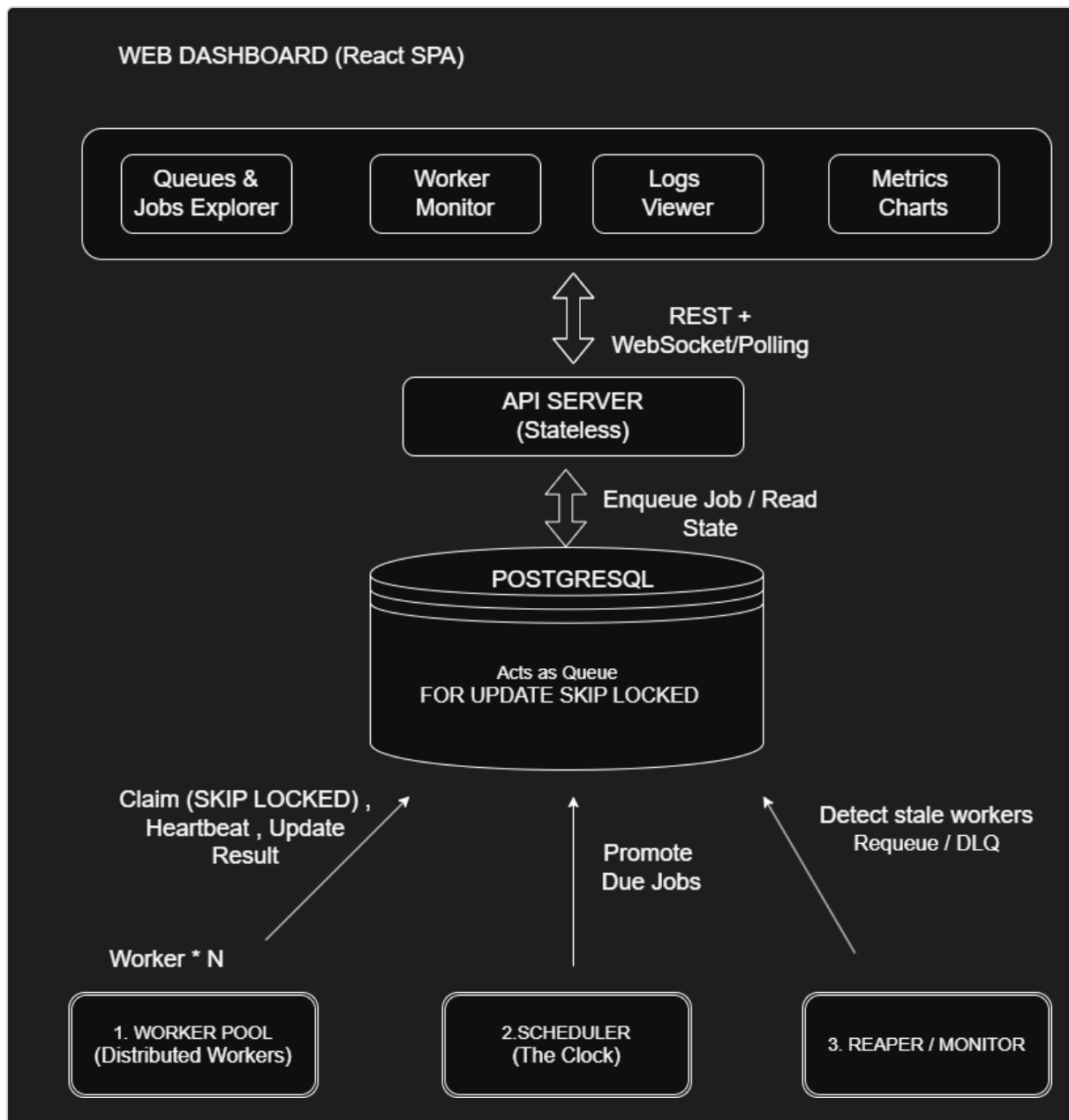


Figure 1. System architecture

The responsibilities of each part are:

- **API server** — handles authentication, projects, queues and job submission, and serves the reads the dashboard needs. It is stateless, so several copies can run behind a load balancer.
- **Scheduler** — the "clock". It promotes delayed and scheduled jobs to the ready state when their time arrives, and turns cron definitions into concrete jobs.
- **Worker** — polls the database, claims jobs atomically, runs them, sends heartbeats, and shuts down gracefully. Adding more workers simply adds throughput.
- **Reaper** — watches for workers that have stopped sending heartbeats, and puts their unfinished jobs back on the queue (or into the Dead Letter Queue if they are out of retries).
- **PostgreSQL** — stores every entity and also acts as the queue.

3. Running each job exactly once

The hardest requirement is making sure that when many workers poll the same queue, no job is ever run twice. I solve this inside the database with a single statement:

```
WITH claimable AS (  
  SELECT j.id FROM jobs j JOIN queues q ON q.id = j.queue_id  
  WHERE j.status = 'queued' AND j.available_at <= now() AND q.is_paused = false  
  ORDER BY q.priority DESC, j.priority DESC, j.available_at ASC  
  FOR UPDATE OF j SKIP LOCKED  
  LIMIT $limit )  
UPDATE jobs j  
SET status = 'claimed', claimed_by = $worker, claimed_at = now(),  
    locked_until = now() + ($lease || ' seconds')::interval,  
    attempt_count = attempt_count + 1  
FROM claimable c WHERE j.id = c.id  
RETURNING j.*;
```

FOR UPDATE locks the candidate rows a worker reads. SKIP LOCKED tells that worker to ignore any row another worker has already locked, rather than waiting for it. So two workers scanning the same queue end up with completely separate sets of jobs — no duplicates and no blocking. Each claim also writes locked_until, a short lease, so that if the worker dies the job does not stay claimed forever.

4. Job lifecycle

A job moves through a small set of states. Immediate jobs start as *queued*; delayed, scheduled and cron jobs start as *scheduled* and are promoted by the scheduler when they are due. Once a worker claims a job it becomes *claimed*, then *running*. On success it is *completed*; on failure it is retried with a backoff delay, and if it runs out of retries it goes to the Dead Letter Queue. Every attempt is stored as a separate row, so the full retry history is kept.

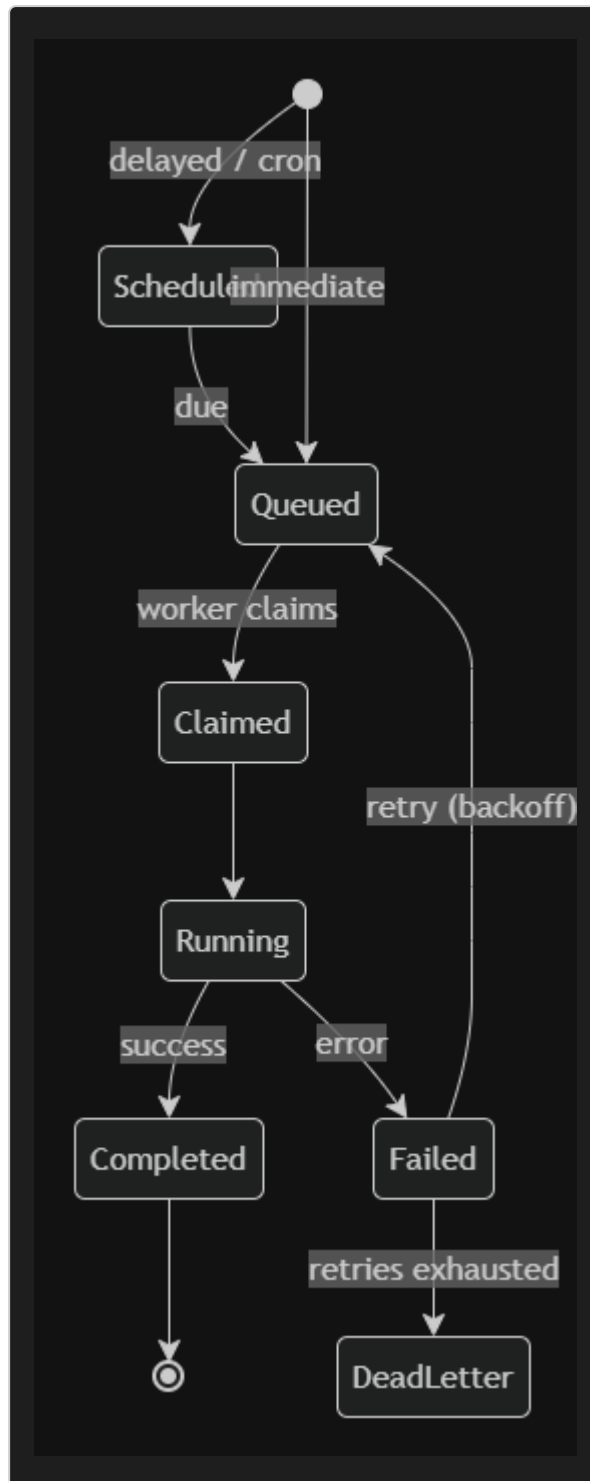


Figure 2. Job lifecycle state diagram

5. Database design

There are 13 tables, kept in third normal form. Most primary keys are UUIDs, which can be generated without asking the database first; the two high-volume append-only tables (job logs and heartbeats) use bigint sequences instead. Ownership flows down a clear tree — organization → project → queue → job — and deletes cascade down that tree, so removing a project cleans up everything under it.

- The index that powers the claim query is *partial* — it only covers rows with `status = 'queued'`, so it stays small even after millions of jobs have completed, and it is pre-sorted in the order the claim needs.
- A partial unique index on `(queue_id, dedupe_key)` stops the same logical job from being submitted twice.

6. Reliability and failure handling

Because workers run on separate processes, the system has to assume any of them can die at any moment. The table below lists the failures I designed for and how each one is handled.

Failure	How it is detected	What happens
A job's task throws an error	the handler rejects	the job is retried with backoff; after the retry limit it moves to the Dead Letter Queue
A worker crashes mid-job	its lease (<code>locked_until</code>) expires	the reaper puts the job back on the queue
A worker silently hangs	its heartbeat goes stale	the reaper marks it dead and reclaims its jobs
The same job is submitted twice	the dedupe-key unique index	the second submission is rejected
A worker is asked to shut down	SIGTERM	it stops taking new work, finishes what it has, then exits

The system gives an *at-least-once* guarantee. In theory a worker could finish a job and then crash before recording the result, in which case the reaper would run it again. Because of this, tasks are expected to be safe to run more than once, and the dedupe key is available for callers who need it.

7. REST API

The API follows normal REST conventions: JWT authentication, JSON request and response bodies, pagination and filtering on list endpoints, and a consistent error shape. The main endpoints are:

Endpoint	Purpose
POST /api/auth/register, /login	sign up / sign in (returns a JWT)
GET, POST /api/projects	list and create projects
GET, POST /api/projects/:id/queues	manage queues (with live counts)
POST /api/queues/:id/pause, /resume	pause / resume a queue
POST /api/queues/:id/jobs, /batch	submit jobs (all five types)
GET /api/projects/:id/jobs	list / filter jobs (paginated)
GET /api/jobs/:id	job detail with its attempts and logs
POST /api/jobs/:id/retry, /cancel	manually retry or cancel a job
GET /api/workers	worker fleet and liveness
GET /api/projects/:id/metrics	dashboard metrics

8. Testing

The tests concentrate on the parts most likely to go wrong under load rather than on trivial CRUD. There are twelve tests and they all pass. The most important one starts twelve workers, has them all claim from a pool of 300 jobs at the same time, and checks that no job was claimed by more than one worker — this is the direct proof that the `SKIP LOCKED` claim is correct. Other tests cover the retry-then-dead-letter path, the three backoff strategies, and the reaper reclaiming jobs from a worker that has died.

Beyond the automated tests, I ran the whole stack under Docker and confirmed end to end that a job submitted through the API is claimed by a worker, executed, and completed, and that when a worker is killed the reaper marks it dead and its work is picked up again.

9. Running the project

The whole system starts with Docker Compose:

```
cp .env.example .env
docker compose up -d --build          # database, api, scheduler, reaper, worker, dashboard
docker compose run --rm migrate npm run seed  # optional demo data
docker compose up -d --scale worker=3 # run more workers

# dashboard: http://localhost:5173 (demo login: demo@example.com / password123)
# api health: http://localhost:4000/health
```


10. Design decisions and trade-offs

Postgres instead of Redis. Using the database as the queue means one fewer system to run and no risk of the queue and the database disagreeing. The cost is that workers poll on a short interval instead of being pushed work instantly, which adds a little latency. For this kind of system that trade is easily worth it, and if push were needed later it could be added with Postgres LISTEN/NOTIFY.

At-least-once, not exactly-once. True exactly-once delivery across crashes isn't really achievable without distributed transactions into whatever the job touches, so instead of pretending otherwise I made repeated execution safe (leases plus an optional dedupe key).

Raw SQL instead of an ORM. The two things this project is really about — the schema and the atomic claim query — are exactly the things an ORM would hide, so I wrote the SQL directly and used a validation library only at the API edge.

What I left out on purpose. Things like WebSocket push, workflow dependencies, rate limiting and queue sharding were listed as bonus features. I chose to leave them out and instead make sure the core — correct claiming, retries, crash recovery — actually works and is tested, rather than adding features that only half-work.